HTTP IN DETAIL

TryHackMe Learning & Practical Report

Prepared by: Sofia

Focus: Web Application Security / WAPT Foundations

Platform: TryHackMe

Room: HTTP in Detail

IqraSec Academy - WAPT Internship - Phase 1

1. Executive Summary

In HTTP in Detail, I went deeper into the protocol: what HTTP and HTTPS stand for, the format of requests and responses, the methods, the status code groups, headers and cookies. At the end I made real requests against a practice site using GET, DELETE, PUT and POST.

2. Lab Information

Platform	TryHackMe
Room / Lab	HTTP in Detail
Primary Domain	Web Application Security
Purpose	Foundational WAPT practice
Completion	Completed - all room tasks finished
Link	https://tryhackme.com/room/httpindetail

3. Learning Objectives

- Understand what HTTP and HTTPS are and how a request/response works.
- Know the main methods and when each is used.
- Recognise common status codes and their meaning.
- Know important request and response headers.
- Understand how cookies keep a session using Set-Cookie.
- Make GET, POST, PUT and DELETE requests to a site.

4. HTTP and HTTPS

HTTP is the HyperText Transfer Protocol and the S in HTTPS means secure: the traffic is encrypted. In the first task I also looked at the practice web page, found the problem on it and clicked it to get the challenge flag.

5. Requests and Methods

The example in the room used HTTP/1.1. The methods I practised: **GET** to view data (a news article), **POST** to create (a new user account), **PUT** to update (change an email address) and **DELETE** to remove (delete an uploaded picture).

6. Status Codes

The room explained the groups: 1xx informational, 2xx success, 3xx redirect, 4xx client error and 5xx server error. Examples I matched in the questions: 201 Created for a new post or user, 404 for a page that does not exist, 503 when the server cannot reach its database or crashes, and 401 when I try to edit a profile without logging in.

7. Headers and Cookies

Request headers such as **Host** (which website is requested) and **User-Agent** (which browser is used) are sent by the browser. Response headers such as **Content-Type** and **Content-Length** tell the browser what kind and how much data comes back. Cookies are saved by the browser after the server sends the **Set-Cookie** header and are sent back with every later request so the server remembers me.

8. Practical Task - Making Requests
GET request to /room
GET request to /blog with the id parameter set to 1
DELETE request to /user/1
PUT request to /user/2 with the username parameter set to admin
POST request to /login with a username and a password in the body
Every request returned a flag, so I could see that each method and parameter changes the answer from the server.

9. Evidence



Fig 1 - HTTP, HTTPS and the challenge flag from the first task.



Fig 2 - HTTP version and Content-Length header.

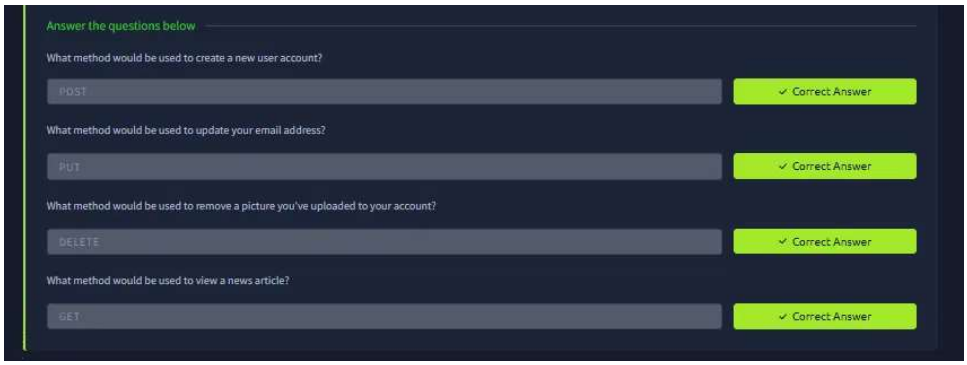


Fig 3 - Choosing the right method: POST, PUT, DELETE, GET.

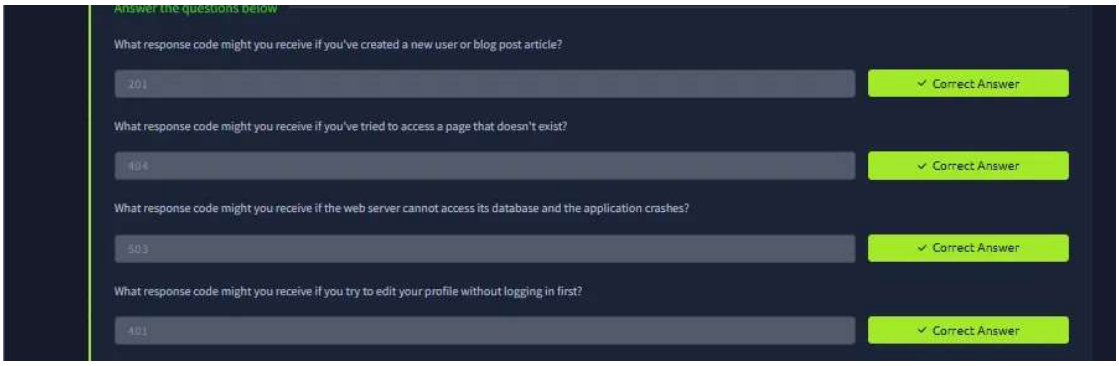


Fig 4 - Status codes matched to real situations.



Fig 5 - Request and response headers.

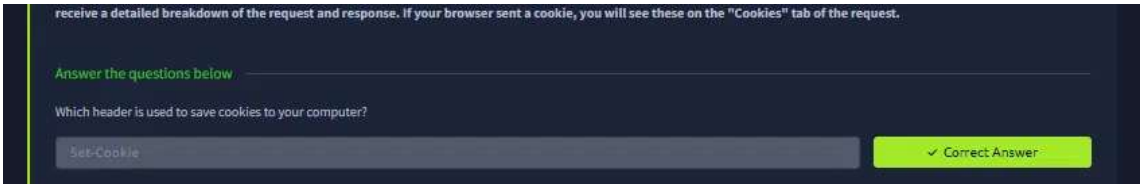


Fig 6 - Cookie question (Set-Cookie).

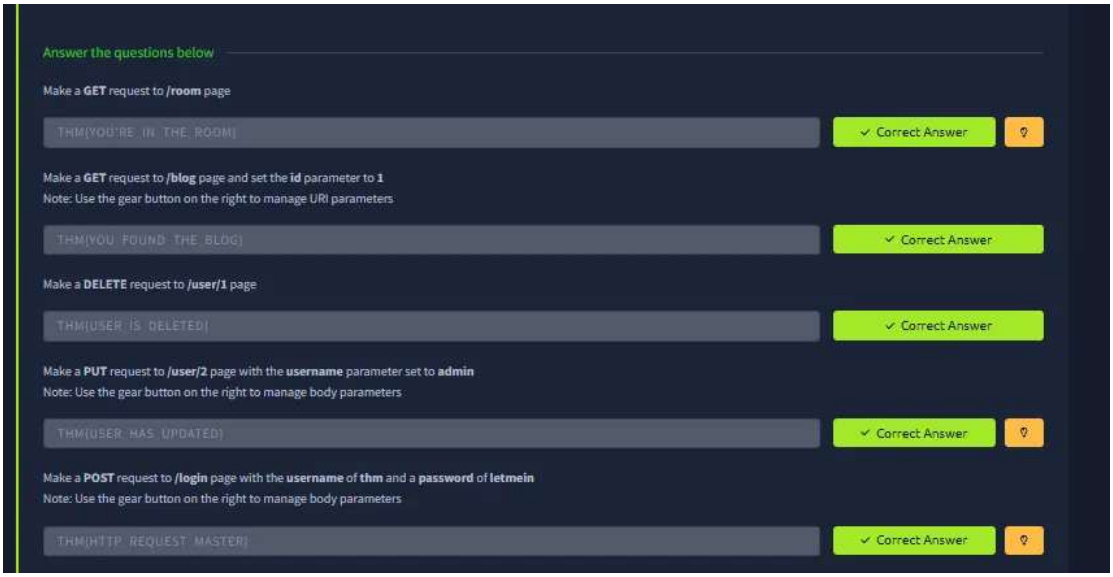


Fig 7 - Practical task: all five requests done.

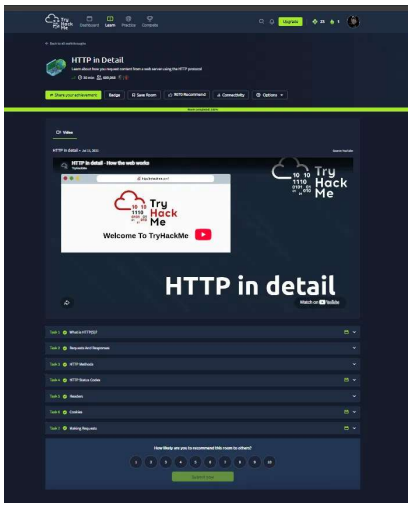


Fig 8 - Room overview with tasks completed.

10. Key Security Takeaways

Choosing the wrong method or leaving PUT/DELETE open to everyone is a serious problem. Status codes tell me a lot about how an application handles my input (401 vs 404 vs 500). Cookies are what keep a session alive, so protecting them (Secure, HttpOnly) is important. Parameters in URL and body can be changed by the user, so they must be validated on the server.

11. Skills Practiced

Area	**Practical understanding gained**
HTTP	Message format and versions
Methods	GET, POST, PUT, DELETE
Status codes	Reading responses by code group
Headers	Host, User-Agent, Content-Type, Content-Length
Cookies	Set-Cookie and sessions
Requests	Sending requests with parameters and body

12. What I Would Explore Next

I will repeat these requests in Burp Suite Repeater and start changing parameters to see how the server reacts.

13. Conclusion

HTTP in Detail made the earlier room clearer. I can now explain every part of a request and a response and I practised sending them with different methods. This will help me a lot in Burp Suite and when testing for vulnerabilities.

Completion status: Completed - all room tasks finished.

14. References

TryHackMe: https://tryhackme.com/room/httpindetail